

はじめに

開発・動作環境

本SDKは、.NET 8.0以上での利用を推奨しています。動作環境および通信要件は以下の通りです。

- 対応OS：
 - Windows 10 (64bit)
 - Windows 11 (64bit)
- 対応フレームワーク：
 - .NET 8.0以上
 - 互換性確保のため、.NET Standard 2.1にも対応
 - ※.NET Frameworkには対応していません。
- 通信要件：
 - 対応インターフェース： Bluetooth 4.1 以上 または USB 2.0 以上
 - Bluetooth 接続時： fNIRSセンサーデバイスが PC と正常にペアリングされている必要があります。
 - USB 接続時： fNIRSセンサーデバイスがPCに接続され、デバイス用ドライバが正常にインストールされている必要があります。

本SDKのダウンロード

本SDKのダウンロードリンクはこちらになります。[ダウンロード](#)

NIRSの基本

信号処理と解析方法

NIRS信号は、脳活動に伴う血液量変化を計測しています。しかし、これには脈波などの脳活動に由来しない信号も含まれます。タスクや刺激提示を開始すると血液量は5〜7秒かけて増加し、止めると15〜25秒（リカバリ期間）かけてベースラインに戻ります。これがタスク/刺激に関係する脳活動に由来した信号成分です。

これ以外に、1 Hz程度（安静時）の脈波、0.3Hz程度の呼吸成分がNIRS信号に現れます。脈波と呼吸の成分は、脳活動に由来する信号に比べて高い周波数を有するため、ローパスフィルタを用いて除去できます。更に、メイヤー波（0.1 Hz付近）などの生理活動に伴う低周波の血液量変動成分も存在します。また、脳は常に活動しているので、タスク/刺激に関係しないバックグラウンド脳活動による信号も観測されます。これらの信号を除去するには、ハイパスフィルタや同期検波の手法が用いられます。一般に周波数フィルタを掛けると、信号波形の両端が歪みます。計測時間を十分長くとれない場合には、この歪の影響が無視できなくなるので、ハイパスフィルタではなく、ベースラインを直線近似して減算するなど別の方法が選択されます。

同期検波とは、前レスト期間 + タスク/刺激期間 + 後レスト期間を1つのブロックとして、これを繰返し、その繰返しに同期して変動する信号成分を抽出する手法です。ここで、注目するタスク/刺激の無い状態をレストと呼びます。前レスト期間はベースライン補正の方法によって適切な時間を決めます。タスク開始時の1点を用いてベースラインとするならばその1点だけでも構いませんが、数秒程度の前レスト期間を設けてベースライン補正に使います。タスク/刺激時間はその種類によってケースバイケースですが、継続したタスクを行う場合には、上記血液の立ち上がり時間より長くとると信号強度が大きくなります。後レストは上記リカバリ時間にベースライン補正用の時間を加えた期間とします。リカバリ時間が十分に取れない場合は、異なる解析法が必要になりますが本SDKでは対応していません。ここで示した時間は、あくまで目安であり、実際に使うタスク/刺激を用いて計測し、適用可能性と要求精度を考慮して定めるようにして下さい。

得られた信号波形をブロック毎に切り出し加算平均すると、繰返しに同期しない信号成分は位相がずれているため重ね合わせにより減少し、同期する成分、つまりタスク/刺激に関連して変動する信号成分を抽出することができます。更に、レスト期間に比べてタスク/刺激期間の信号が有意に変化しているかどうかを調べるには、平均値差検定を行います。繰返し計測を行わない場合でも、NIRS信号はその計測原理上、ある時点の値からの増量または減少量しか得られませんので、必ず信号の原点を決める必要があります。装置出力は、便宜的に計測開始時を0とすることが多いですが、前述のようにベースラインのゆらぎが大きいため、注目する脳活動を生じさせるタスクまたは刺激の開始前の値または時間平均値を基準に取り直すことが一般的です。High-Level APIは、上記信号前処理と同期検波に対応した信号処理および解析を行なう機能を有します。これまで信号解析は計測後に行うことしかできませんでしたが、本APIを用いることにより、計測中に解析を実施して、その結果を用いて次の計測条件を選択するなど、リアルタイム性を備えたアプリを容易に作成することができます。

本SDKでは、NIRS信号を取得するためのLow-Level APIとNIRS信号を適切に扱うためのHigh-Level APIを用意しています。

ExBrain SDK Low-Level API 解説書

はじめに

この文書は、プログラミングや脳機能計測の専門知識がない方でも、ExBrain SDK のLow-Level APIが **何をしているのか／どうやって実現しているのか** を理解できるように、平易な言葉で解説するものです。

ExBrain SDK は、NeU 製の小型脳活動センサ（額に装着する HOT-2000 や XB-01 など）をパソコンから操作するための「部品集」です。Low-Level APIは、その中でも **センサとの通信・データの受け取り・数値への変換** という、最も土台になる部分を担当します。

1. このセンサは何を測っているのか

額に当たる部分に、赤外線に近い光を出す「光源」と、戻ってきた光を受け取る「受光素子（フォトダイオード、略してPD）」が並んでいます。

- 光は皮膚や頭蓋骨を通して脳の表面（前頭前野）まで届き、一部が戻ってきます。
- 血液中のヘモグロ빈は、この光を吸収します。
- **脳が活発に働くと血流が増え、ヘモグロ빈の量が変わる** ため、戻ってくる光の量も変わります。その量は、ヘモグロ빈濃度と光路長（光が辿った距離）の積の変化量（dTHbL）に比例します。（計測原理上、ヘモグロ빈濃度の絶対値を求めることはできません。）

センサは左右それぞれに、光源から **1cm 離れた受光素子** と **3cm 離れた受光素子** を持っています。近い方（1cm）は主に皮膚表面の血流を、遠い方（3cm）はより深い脳の血流を反映します。「**3cm の信号 - 1cm の信号**」を計算して表面の影響を差し引き、さらに脈拍や呼吸による変動を取り除いて **脳活動の指標（ブレインインデックス）** を求めます。

この「光の戻り量」から「ヘモグロ빈濃度×光路長の変化量（dTHbL）」を計算し、さらに「脳活動の指標」や「脈拍数」を求めるのが、低レベル層の中心的な仕事です。

2. 全体の流れ

Low-Level APIを使うと、おおむね次の順序で処理が進みます。

- ① センサを探す（探索）
↓
- ② センサにつなぐ（接続）
↓
- ③ 装着の具合を整える（装着調整）
↓
- ④ 基準値を取る（キャリブレーション）
↓
- ⑤ 測る（計測） — データがどんどん届く



⑥ 終わる（切断）

それぞれの段階を順に見ていきます。

3. 各機能のしくみ

3.1 センサを探す（デバイス探索）

パソコンの近くにあるセンサを見つける機能です。USB ケーブルでつないだもの、Bluetooth で過去にペア設定したもの、まだペア設定していないもの、をそれぞれ対象にするか選べます。[コード：探索条件は `ExBrainSearchParameter` の `ShouldUSB` / `ShouldPairedBluetooth` / `ShouldNonPairedBluetooth`（各 `bool`）で指定]

見つかったセンサは「追加されました」「情報が変わりました」「消えました」といった**お知らせ（コールバック）**でアプリ側に随時伝えられます。たくさんのセンサがある環境では、**名前の一部（キーワード）で絞り込む**こともできます。[コード：絞り込みは `ExBrainSearchParameter.Keywords`（文字列配列、複数可）]

3.2 センサにつなぐ（接続）

探索で見つけたセンサを選んで接続します。接続すると、まずセンサから **機種・固有番号・ファームウェアの版・各種設定**といった情報を読み取ります。

- センサの古いファームウェア（2019 年 4 月版より前）は対応していないため、接続を中止します。
- センサ本体が独自に脈拍計算などをする設定になっていると、SDK 側の処理と二重になるため、接続時にその機能をオフにします。
- 通信が突然切れても、設定によっては**自動でつなぎ直**します。さらに、計測中に切れて再接続できた場合は、**計測も自動で再開**します。

接続中・計測中・切断中といった状態は、常にアプリ側へ通知されるので、利用者に「いま何をしているか」を画面表示できます。

3.3 装着の具合を整える（装着調整 / ゲイン調整）

センサを額にどう当てているか（髪の毛の量、肌の色、押し当て具合）によって、戻ってくる光の強さは大きく変わります。光が弱すぎても強すぎても、良いデータは取れません。

そこで、受光素子の**増幅率（ゲイン）を自動で調整**します。

- 信号が**弱すぎる**（基準値より小さい）→ 増幅率を上げる
- 信号が**強すぎる**（飽和している）→ 増幅率を下げる
- ちょうど良くなるまで、少し変えては測り直す、を繰り返す

ちょうど良い範囲に収まったら、最後に**目標値ぴったりへ微調整**し、もう一度確認して仕上げます。何度繰り返しても収まらない場合は「これ以上調整できません」とエラーで知らせます。通常、このエラーは装着状態を改善することで回避で

きます。調整の経過は `MGCPParam.csv` というファイルに記録することもできます。

このゲイン調整は、装着後に必ず実施してください。

3.4 基準値を取る（キャリブレーション）

人や装着状態によって、計測値には個人差の「下駄（ゲタ）」が乗っています。計測開始の最初の十数秒（標準で約 12.8 秒）のあいだ静かにしてもらい、その平均を**基準（ゼロ点）**として記録します。以降のデータからこの基準を差し引くことで、**変化分だけ**を正しく見られるようにします。

一度取った基準は使い回すことも、毎回取り直すこともできます。[コード：計測開始時の `recalibrate` フラグ（`false` = 既存を再利用／`true` = 取り直し）]

3.5 測る（計測）

センサからは毎秒10 回、次のような生データが届きます。

- 4つの受光素子（HOT-2000 左右 × 1cm/3cm）の信号と飽和度（XB-01は2つの受光素子）
- 加速度・角速度（体の動き）
- バッテリ残量・電圧

Low-Level APIは、これらがすべて揃ってから **1 件分のデータとして確定**し、以下の加工を施してアプリへ届けます。

1. **ヘモグロビン濃度**に変換（光の戻り量を対数計算し、係数をかけて濃度に直す）
2. 基準値を差し引く（キャリブレーション）
3. **ブレインインデックス**を計算: (3cm - 1cm) の信号を平滑化（後述の「フィルタ」）
4. **脈拍数**を計算
5. **脳活動度（カラーインデックス）**を計算
6. 体動などによる**ノイズの有無**を判定

計測データはアプリへ渡すと同時に、CSV ファイルへ保存することもできます。

3.6 ノイズの扱いと「フィルタ」

計測データには、まばたき・体の動き・電氣的なゆらぎなど、さまざまな**雑音**が混じります。これを和らげるのが「フィルタ」です。本 SDK では主に次を使います。

- **メディアンフィルタ**：直近数件を小さい順に並べ、**真ん中の値**を採用する。突発的なトゲ（外れ値）に強い。
- **移動平均**：直近数件の**平均**をとってなめらかにする。
- **最大／最小フィルタ**：直近数件の最大・最小を見る。値の暴れ具合の検出に使う。

たとえば、ある信号の一定期間の「直近の最大と最小の差」が大きすぎるときは、体が動いたとみなして `NoiseDetectionFlag`を付けます。

3.7 脈拍数の求め方

額のdHbTLデータには、**心臓の鼓動に合わせた細かな波（脈波）**が含まれています。ここから脈拍数を割り出します。手順は次の通りです。

1. 一定時間分（約 6.4 秒分）のデータをためる。
2. 体が動いた区間は信頼できないので**印を付け、近くの良い値で埋める**。
3. **周波数の分解（FFT）**という数学的な手法で、脈拍に相当する波（毎分 48～180 回の範囲）だけを取り出す。
4. 取り出した波が**プラスとマイナスを横切る回数（ゼロクロス）**を数え、1 拍にかかる時間を逆算して **1 分あたりの拍数**を求める。

複数の受光素子から別々に推定し、極端な値を避けるため**中央寄りの値**を採用します。

3.8 脳活動度（カラーインデックス）

「脳が活発／落ち着いている」を、おおまかな**段階（メーター）**で表す指標です。ブレインインデックスが上がり続けているか／下がり続けているかの傾向を 時間をかけて集計し、急に跳ねないようにゆっくり追従する数値として表現します。一方向に偏りすぎた場合は自動的に中庸の値へ戻す仕組みも入っています。

3.9 その他

- センサは、6軸の加速度計を持っていて、頭部の動きを表す加速度(Acc)と角加速度(Gyro)が送られてきます。現在は、標準モード（1 秒間に10回計測）のみに対応しています。
- 脳活動量の変化の基準となる期間や、注目する期間を設定して、各計測データにマークを付けることができます。
- 信号が振れすぎて正しい値が得られなかった場合に、Saturationフラグを付けます。
- センサのバッテリー状態を取得します。

4. 「裏側で待たずに動く」しくみ

センサとの通信は、命令を送って返事を待つ、という往復が何度も発生します。これを画面操作と同じスレッド（処理の流れ）で行うと、返事を待つあいだ **アプリが固まってしまう**。

そこで本 SDK は、通信や計測やファイル保存を**専用の裏方スレッド**に任せ、アプリ側は固まらずに動き続けられるようにしています。さらに、「命令を送る → 返事を待つ → 次の命令」という一連の流れを、待ち時間に 他の処理を譲りながら順番にこなす**タスク（コルーチン）**という仕組みで 記述しています。これにより、**複雑な手順を読みやすく・安全に**実行できます。

複数の裏方スレッドが同じデータに同時に触れて壊さないよう、共有データには ****鍵（排他制御）****をかけて、一度に 1 つの処理だけが触れるようにしています。

5. エラーと安全性

- すべての操作は、成功・失敗を**結果コード**（例：成功、未接続、パラメータ不正、ライセンスエラーなど）で返します。途中で異常が起きても、アプリ全体が **落ちないように設計**されています。[コード： `EnmSdkResult` (`eSuccess`

= 成功 / `eNotConnected` = 未接続 / `eParamWrong` = パラメータ不正 / `eLicenseError` = ライセンスエラー など)]

- 利用には**ライセンス**が必要で、無い場合はエラーで知らせます。
 - 接続を終えるときは、裏方スレッドの後始末を確実に行ってから、ログファイルを閉じます。
-

6. まとめ

Low-Level APIは、脳活動センサとの**「会話」**を一手に引き受ける土台です。

- センサを**探し・つなぎ・装着を整え・基準を取り・測る**、という一連を担う。
- 光の戻り量をdTHbLへ、さらに**脳活動の指標・脈拍・活動度**へ変換する。
- 雑音を**フィルタ**で抑え、必要なら**CSV 保存**する。
- すべてを**裏方スレッド**で処理し、アプリを固まらせない。

ここで作られた「きれいな計測データ」を受け取り、実験の進行管理や統計解析を行うのが、上位の **High-Level API** です（別冊の High-Level API 解説書を参照）。

ExBrain SDK High-Level API 解説書

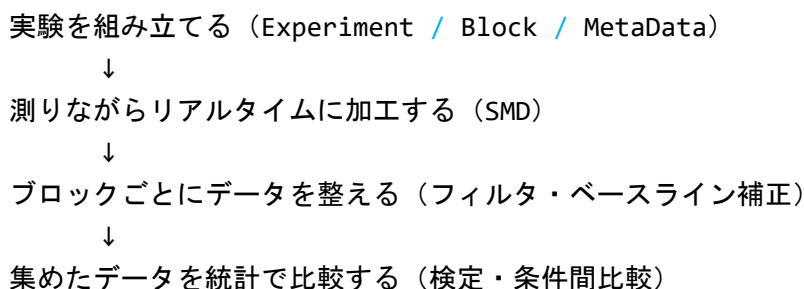
はじめに

この文書は、プログラミングや統計、脳機能計測の専門知識がない方でも、ExBrain SDK の「High-Level API」が **何をしているのか／どうやって実現しているのか** を理解できるように、できるだけ平易に解説します。

Low-Level APIが「センサと会話してきれいな計測データを作る」土台だったのに対し、**High-Level APIは「実験を進行させ、結果を統計的に分析する」**役割を担います。脳活動の実験（たとえば「ある課題をやっているときと安静時で脳活動に差があるか？」）を行う研究者・実験者を支える層です。

統計の部分は本 SDK の中でも特に専門性が高いところなので、**身近なたとえ**を交えて丁寧に説明します。

1. High-Level APIが実現することの全体像



順番に見ていきます。

2. 実験を「ブロック」で組み立てる

心理・生理の実験は、ふつう同じ手順を何度も繰り返します。本 SDK では、その 1 回分を「ブロック」と呼び、ブロックを 4 つの段階に分けて管理します。

- ① 事前ベースライン補正(preFitting) ... 課題前の安静・準備
- ② 課題(task) ... 計算問題を解く、など実際の課題
- ③ 安静(calmDown) ... 課題後の落ち着き時間
- ④ 事後ベースライン補正(postFitting) ... 締めめの安静

各段階を**何秒にするか**は「MetaData」というテンプレートで決めておきます。あとは「ブロックを実行」と指示すれば、SDK が**時間を計りながら自動で段階を進め**、それぞれの境目に****目印（マーカー）****をデータへ書き込みます。これにより、あとから「ここからが課題区間」とはっきり区別できます。

複数のブロックには、**被験者の識別子**や**繰り返し番号**を付けられるので、「誰の・何回目の・どの段階のデータか」を正確に管理できます。

3. 測りながら加工する（リアルタイム処理）

3.1 注目する量（関心量）

とくに注目するのは、「3cm の信号 - 1cm の信号」（dTHbL）です。Low-Level API 解説書で述べたとおり、これは皮膚表面の影響を差し引いた、より脳に近い活動の指標です。

3.2 デジタルフィルタ（なめらかにする）

計測値には細かなゆらぎや、目的外の速い／遅い変動が混じります。これを取り除くのが **デジタルフィルタ** です。本 SDK では **Butterworth**（バターワース）フィルタという、音響機器などでも広く使われる 定番のフィルタを用意しています。

- **ローパス**：速すぎる変動（高周波の雑音）を削り、ゆっくりした本質的な変化を残す。
- **バンドパス**：「速すぎる」「遅すぎる」の両方を削り、ちょうど見たい速さの変化だけを残す。

本 SDK では、ブロックの計測が終わった時点で、そのブロック全体にまとめてローパスフィルタをかけます。しかも前向き・後ろ向きの二重がけを行うことで、「時間のずれ（位相の遅れ）が出ない」方式を採用しています。これにより、波形の山や谷の位置がずれず、課題に対する反応のタイミングを正確に保ったままなめらかにできます。

3.3 SMD（標準化平均差）——「基準と比べてどれだけ違うか」

実験では、「ある基準の状態に比べて、いま脳活動がどれくらい変化したか」を知りたいことがよくあります。これを数値化するのが **SMD**（標準化平均差）です。

身近なたとえで言うと——

あるクラスのテストで、A さんが 70 点を取りました。これは「良い」のでしょうか？ それは**クラスの平均点とばらつき**によります。平均 50 点・みんなの点がバラバラなクラスでの 70 点と、平均 68 点・みんな僅差のクラスでの 70 点とでは、「目立ち具合」がまるで違います。

SMD は、この「ばらつきを物差しにした、目立ち具合」を表す数値です。具体的には、

$$\text{SMD} = (\text{いまの平均} - \text{基準の平均}) \div \text{全体のばらつき (標準偏差)}$$

として計算します。割り算で「ばらつき」を物差しにしているので、**単位やスケールによらず公平に「効果の大きさ」を比べられる**のが利点です。（統計学では Cohen's d と呼ばれる考え方で、本 SDK ではサンプルが少ないときの 偏りを補正する工夫も入れています。）

ただし、生体信号は自己相関と呼ばれる自分自身との類似性が高いため、統計学で用いられる効果量より値が大きくなることに注意してください。また、信号の立上りや立下りなど急峻な変化は「ばらつき」を大きくするため、SMD は減少する方向に働きます。

使い方は次のとおりです。

1. まず **基準区間**（リファレンス）を設定します（例：安静状態を一定時間記録）。SDK はこの区間の**平均**とばらつきを覚えます。[コード：開始は `StartReferencePeriod(name)`、確定は `FinishReferencePeriod()`]
2. その後の計測では、**直近の一定区間**と基準区間を比べて SMD を計算し、リアルタイムにアプリへ通知します。[コード：「直近の一定区間」のフレーム数は `Experiment` 生成時の `smdTargetWindowSize`]

これにより、「いま、基準よりどれだけ脳活動が安定的に高まっているか」を**その場で**把握できます。

4. ブロックを整える（ベースライン補正）

ゆっくりした血液量変動のため、データ全体が**ゆっくり右肩上がり**（または**右肩下がり**）に流れていくことがあります。これを「ドリフト（基線変動）」と呼びます。ドリフトは、課題遂行以外の脳活動や全身性の血液量変化により発生します。このまま課題区間を見ると、課題の効果なのかドリフトなのか区別できません。

そこで本 SDK は、**課題の前後にある安静的な区間**（事前ベースライン補正区間・事後ベースライン補正区間）に注目します。この 2 区間は本来「課題遂行による脳活動変化が無い」はずなので、ここを**結ぶまっすぐな傾きの線**を引いて「これがドリフトの正体」と推定し、**ブロック全体からその傾きを差し引きます**。

たとえるなら、傾いた机の上で測ったグラフを、机の傾きぶんだけ**水平に直す**作業です。課題区間と課題終了後に血液が元にもどるまでの期間は傾きの推定に使わない点がポイントで、課題の効果をドリフトと取り違えないようにしています。

5. 統計で比べる —— 「その差は意味があるのか？」

実験データが集まったら、いよいよ分析です。ここが High-Level API の最も専門的な部分です。

5.1 そもそも「統計的に比べる」とは

たとえば「課題中の脳活動は安静時より高い」と言いたいとします。たまたま高い値が出ただけかもしれません。「**その差は、偶然では説明しにくいほど大きいのか？**」を判断するのが**統計的仮説検定**です。

検定の結果は **p 値**（ピーチ）という数値で出ます。ざっくり言うと、

「**p 値** = 「本当は差が無いのに、たまたまこれくらいの差が出てしまう確率」

です。p 値が小さい（慣習的に 0.05 未満）ほど、「偶然とは考えにくい = **本物の差らしい**」と判断します。本 SDK の各検定は、この **p 値**を計算して返すのが役目です。

5.2 用意されている検定の種類

データの性質に応じて、適切な検定は変わります。本 SDK は主要な検定を揃えています。

検定	コード（C# 直接呼び出し）	どんなときに使う（ざっくり）
Student の t 検定	<code>Statistics.StudentT</code>	2 グループの平均を比べる。データが釣鐘型で、ばらつきが同程度
Welch の t 検定	<code>Statistics.WelchT</code>	同上だが、2 グループのばらつきが違うとき
対応のある t 検定	<code>Statistics.PairedT</code>	同じ人の「前」と「後」など、ペアになったデータを比べる
Mann-Whitney U 検定	<code>Statistics.MannWhitneyU</code>	釣鐘型でなくてもよい。2 グループを順位で比べる
Wilcoxon 符号順位検定	<code>Statistics.WilcoxonSignedRank</code>	ペアのデータを、釣鐘型を仮定せず順位で比べる
Shapiro-Wilk 検定	<code>Statistics.ShapiroWilk</code>	データが釣鐘型（正規分布）かを調べる
Levene 検定	<code>Statistics.Levene</code>	2 グループのばらつきが同じかを調べる

ここで「釣鐘型（正規分布）」とは、平均の周りに山なりに分布する、自然界でよくある形の ことです。t 検定はこの形を前提にしますが、現実のデータがそうでないこともあります。そのときは順位を使う Mann-Whitney / Wilcoxon（ノンパラメトリック検定）が頑健です。

5.3 「平均」ではなく「順位」で比べるとは

Mann-Whitney や Wilcoxon は、値そのものではなく順位を使います。たとえば 2 グループの数値を全部いっしょに小さい順に並べ、「グループ A は上位に固まっているか、下位に固まっているか」を見ます。A が上位ばかりなら「A の方が大きい」と言えます。外れ値（極端な値）に振り回されにくいのが長所です。

5.4 計算の中身（数学が好きな方向けの補足）

専門外の方は読み飛ばして構いませんが、内部では次のような数学的部品を使っています。

- **正規分布・t 分布・F 分布の確率計算**：これらの「確率の面積」を求めるために、**正則化不完全ベータ関数**という関数を**連分数**という方法で精密に計算します。
- **正規分布の逆計算**（ある確率に対応する値を求める）には、**有理式による高精度近似**を使います。
- **Wilcoxon 検定**では、データが少ないときは**すべての可能な並び方を数え上げて 厳密な確率**を出し、多いときは**正規分布で近似**します。さらに同じ値が複数あるとき（同順位）の**補正**も入れています。
- **Shapiro-Wilk 検定**は、Royston という研究者が提案した近似式に基づいて、データが釣鐘型からどれだけ外れているかを数値化します。

これらはいずれも、**専用の統計ソフトと同等の結果**を、SDK 単体で出すための実装です。

5.5 検定の自動選択

「どの検定を使えばいいかわからない」という方のために、本 SDK は **検定を自動で選ぶ**機能を持っています。判断の流れは、統計の教科書どおりです。

```
まず Shapiro-Wilk で「データは釣鐘型か？」を判定
├─ 釣鐘型でない → 順位の検定 (Mann-Whitney / Wilcoxon)
└─ 釣鐘型である → 次へ
    Levene で「2 グループのばらつきは同じか？」を判定
    ├─ 同じ → Student の t 検定
    └─ 違う → Welch の t 検定
(同じ人のペアデータなら → 対応のある検定を選ぶ)
```

利用者は「お任せ（自動）」を選ぶだけで、データに合った妥当な検定が使われます。[コード：比較関数の `isParametric` / `isEqualVariance` を未指定（C# では `null`）にすると自動判定] もちろん、検定を**自分で指定**することもできます。[コード： `isParametric` / `isEqualVariance` に `true/false` を明示するか、個別の検定関数（`Statistics.WelchT` など）を直接呼ぶ]

5.6 すぐ使える「条件間の比較」メニュー

実際の研究でよくある比較を、そのまま呼べる**関数**として用意しています。利用者は細かい統計手続きを意識せず、目的に合うものを選ぶだけです。

比較メニュー	コード（C# 関数）	何を比べるか
被験者内・課題間比較	<code>TaskComparison.CompareWithinParticipantTasks</code>	同じ人が行った 2 種類の課題 の脳活動を比べる
被験者内・事前事後比較	<code>TaskComparison.CompareWithinParticipantPrePost</code>	同じ人の 訓練前と訓練後 などを比べる
対応のある群課題比較	<code>TaskComparison.ComparePairedGroupTasks</code>	ペア対応のある 2 群 を比べる
群間・課題比較	<code>TaskComparison.CompareBetweenGroupsTask</code>	別々の人 からなる 2 グループ を比べる
被験者間・課題比較	<code>TaskComparison.CompareBetweenParticipantsTasks</code>	異なる被験者 集団 どうしを比べる
課題対ベースライン比較	<code>TaskComparison.CompareTaskAndBaseline</code>	課題区間 と 事前安静区間 を比べる（課題の効果を見る）

それぞれの比較では、正しい使い方を守っているかも自動でチェックします（例：「被験者内比較なのに別人のデータが混ざっていないか」「ペア比較なのに 数が合っているか」など）。間違いがあればエラーで知らせます。

結果は、「使った検定の種類」「p 値」「グループ間の差」が返ります。左の計算でつまずいても右の計算は止まらないなど、頑丈な作りになっています。

6. 裏側の安全設計（簡単に）

- 計測データは測りながら次々と書き足され、同時に分析側から参照されます。この同時アクセスでデータが壊れないよう、**鍵（排他制御）**で守っています。
- 各統計計算で異常（例：データが少なすぎる）が起きても、アプリ全体が落ちないように、エラーは数値や結果コードに変換して安全に返します。

7. まとめ

High-Level APIは、脳活動の実験そのものを支えるAPIです。

- 実験をブロックと段階で組み立て、時間管理と目印付けを自動で行う。
- 測りながらSMDで「基準からの変化」をその場で数値化する。
- ブロックごとにフィルタ（位相のずれない二重がけ）でなめらかにし、ベースライン補正でドリフトを取り除く。
- 集めたデータを、用途別の比較メニューと自動で選ばれる統計検定で分析し、「その差に意味があるか（p 値）」を判定する。

これらにより、脳機能計測や統計の専門家でなくても、脳活動データから意味のある結論を引き出せるよう設計されています。これまでは、計測後に解析を行うことが一般的でしたが、本SDKを用いれば、計測途中で解析をして、その結果を後の計測に活かすというような、高度なアプリを簡単に作成できます。土台となる計測のしくみについては、別冊のLow-Level API 解説書を参照してください。

NeUライセンスマネージャー

はじめに

本SDKを利用するには発行されたライセンスを受け取り、NeU ライセンスマネージャーによるライセンス有効化が必要になります。

ライセンスが有効化されていない状態で本SDKを利用しようとすると、例外処理が返ってくるような仕様になっております。

NeUライセンスマネージャーの概要

NeU ライセンスマネージャーは、NeU 製品のライセンスを管理するための Windows デスクトップアプリです。複数のライセンスを同時にインストールして使用でき、各ライセンスの状態確認や、有効化・再有効化・無効化を行うことができます。

本ツールでは以下の操作が可能です：

- インストール済みライセンスの一覧表示
- ライセンスの新規有効化
- 既存ライセンスの再有効化（サーバーとの同期）
- ライセンスの無効化
- ライセンス状態の日本語表示
- 手動の再読み込みによる最新状態への更新 ライセンスに関する処理はすべて NeU ライセンスシステムを通して行われます。

主な機能

複数ライセンスの同時管理

複数のライセンスがインストールされている場合、それらを一覧で確認できます。

サーバー連携

有効化・再有効化・無効化は NeU ライセンスサーバーと通信して行われます。

ローカルライセンス情報の確認

ローカルに保存されているライセンス情報（有効期限、GUID、名義など）を表示できます。

ライセンスの内容

一覧に表示される各ライセンスには、以下の情報が含まれます。

ライセンス名義

ユーザーまたは企業名

GUID

ライセンス固有の識別子

有効期限

「無期限」または指定された日付（YYYY/MM/DD）

状態

有効期限切れ/破損/ハードウェア不一致/PC の時刻が正しくありません/その他/不明（英語名が表示されます）

ユーザーインターフェース

画面は以下のように構成されています。

NeUライセンス管理 v1.0

ライセンスタイプ: Basic

ライセンス名義: test1

GUID: 52cd9fd4-0c27-4e4e-a24a-a2653815ffce

有効期限: 2026/12/31

状態: 有効

再有効化

無効化

再読み込み

新しいライセンスを有効化

再読み込み

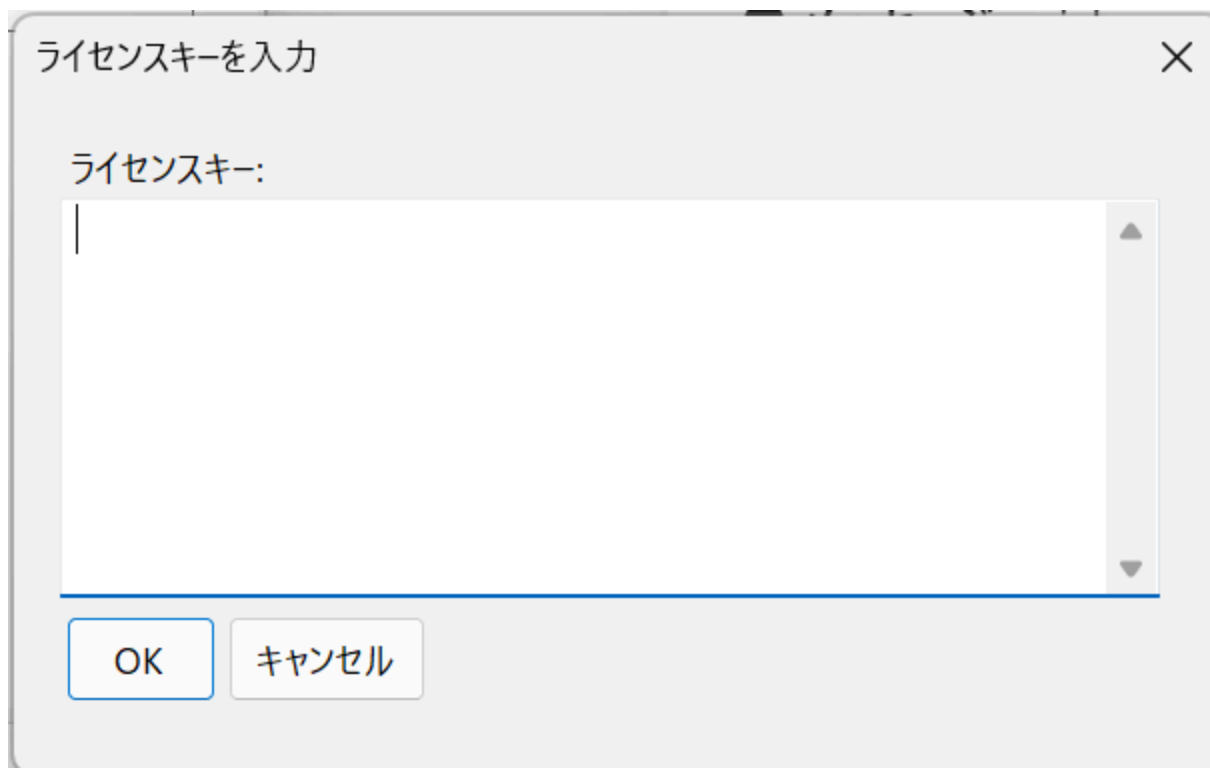
ライセンス情報を再読み込みし、画面を最新状態に更新します。以下の場合に使用します：

- 外部ツールでライセンスを変更した
- 有効化・無効化後に画面が更新されていない
- 状態を手動で確認したい

新しいライセンスを有効化

ライセンスキー入力ダイアログが開き、入力されたキーをサーバーに送信して以下を行います：

- ライセンスの登録（新規・更新）
- ローカルストアへの保存
- 成功またはエラーの表示
- 一覧の自動更新



再有効化 (各ライセンス上に表示)

サーバーとライセンス情報を再同期します。次の場合に使用します：

- システム時刻を修正した後
- サーバー側の設定が更新された
- ライセンスの状態がおかしいと感じた場合
- 有効期限の延長などが行われた場合

無効化 (各ライセンス上に表示)

ライセンスをローカルから削除し、サーバー上でも無効化します。次のような場合に使用します：

- PC を変更する
- ライセンスを他のユーザーに移したい
- 端末からライセンスを削除したい
- 再度有効化することも可能です（契約内容に依存します）。

リリースライセンス パッケージングガイド

このページは、ExBrain SDK アプリをリリース向けに準備するアプリケーション開発者向けのドキュメントです。

重要なルール

リリースライセンスを作成する際は、ビルドフォルダーではなく、インストール済みのアプリのコピーを NeU License Manager で指定してください。

インストール済みフォルダーがお客様の実行環境です。ビルドフォルダーには余分なファイル、インストーラー出力の欠落、一時バイナリ、デバッグシンボル、または異なるレイアウトが含まれていることがよくあります。誤ったフォルダーを選択した場合、リリース SDK はアプリを **AppInvalid** として拒否する可能性があります。

リリースフロー

1. アプリケーションをビルドします。
2. お客様と同じように、ローカルでインストーラーを実行します。
3. NeU License Manager を開きます。
4. **Create Release License** を選択します。
5. インストール済みアプリケーションフォルダー、またはインストール済みの EXE/スクリプトエントリポイントを選択します。
6. 生成されたリリースパッケージファイルの保存先フォルダーを選択します。
7. NeU License Manager はインストール済みファイルを確認し、ハッシュとアプリ検証 JSON の両方を含む **AppHash** ペイロードを作成します。
8. ツールはその **AppHash** ペイロードを NeU ライセンスサーバーに送信します。
9. サーバーはその **AppHash** ペイロードを含む署名済みリリースライセンスを返します。
10. インストーラーを更新して、生成されたリリースライセンスをレジストリにインストールします。

このプロセスは、出荷するアプリケーションファイルを変更するたびに繰り返してください。

Python アプリ

Python アプリケーションの場合は、出荷するスクリプトとネイティブ依存関係が含まれるインストール済みフォルダーを選択するか、インストール済みエントリスクリプトを選択してください。

実行時は、インストール済みアプリフォルダーを作業ディレクトリとしてアプリを起動するか、そのアプリフォルダーに **ExBrainSdkNative.dll** を配置するか、SDK を読み込む前に **NEU_APP_ROOT** をインストール済みアプリフォルダーに設定してください。これにより、SDK が署名済みリリースライセンスにリストされたファイルをチェックするためのルートフォルダーが提供されます。

リリースアプリのチェックには次のようなファイルが含まれます：

- **.py**、**.pyw**、**.pyz**
- **.pyd**
- **.dll**

- パッケージデータファイル
- アプリと一緒に出荷されるロック/設定/リソースファイル

`__pycache__` や `.pyc` ファイルなどの生成されたキャッシュは除外されます。`log`、`logs`、`temp`、または `tmp` という名前の一般的なトップレベルの可変フォルダーは除外されます。必要な出荷アセットをこれらのフォルダーに入れないでください。

生成ファイル

NeU License Manager は選択した出力フォルダーに次のファイルを書き込みます：

- `NeUReleaseLicense_<license-guid>_instructions_and_license.txt`：バイリンガルのインストーラー手順とリリースライセンス。

この手順ファイルには完全なリリースライセンスが含まれています。インストーラーはそのライセンステキストを示されたとおりに正確に書き込む必要があります。

インストーラーのレジストリキー

エンドユーザー向けリリースビルドの場合は、マシン全体のレジストリ場所にリリースライセンスをインストールしてください：

`HKLM\Software\NeU\Licenses`

次の文字列値を作成します：

- 値の名前：リリースライセンスの GUID
- 値のデータ：手順ファイルの完全なリリースライセンステキスト

`HKLM` への書き込みには管理者権限が必要です。アプリ開発者はアプリケーションインストーラーからこの値を書き込む必要があります。

個別の JSON ファイルや個別の `AppHash` レジストリキーは不要です。アプリ検証 JSON は署名済みリリースライセンスに埋め込まれています。

NeU License Manager を通じてアクティベートされた開発者ライセンスは、引き続き `HKCU\Software\NeU\Licenses` にユーザーごとにインストールされます。SDK は `HKCU` と `HKLM` の両方を検索します。

AppInvalid が発生する原因

リリース SDK は次の場合にライセンスを `AppInvalid` として報告します：

- 埋め込まれたアプリ検証 JSON が存在しない
- 埋め込まれたアプリ検証 JSON の `appHash` が署名済み `AppHash` ペイロードハッシュと一致しない
- 埋め込まれたアプリ検証 JSON が無効なスキーマに編集された
- リストされたファイルが欠落している

- リストされたファイルのサイズが異なる
- リストされたファイルの SHA-256 コンテンツが異なる

SDK は **AppHash** を確認する前に一貫した内部表現を再構築するため、ホワイトスペースと JSON プロパティの順序は問題ありません。

オフライン動作

通常のリリース SDK 機能チェックはインターネットアクセスを必要としません。リリースライセンスは開発者やエンドユーザーのマシン ID に紐付けられていません。インストール済みアプリが署名済み **AppHash** と一致する場合にのみ、任意のマシンで有効です。

エンドユーザーのツールは後から再アクティベーションリクエストを送信することができます。これにより、エンドユーザーの PC をサーバーに表示させることができ、アプリまたはライセンスマネージャーがサーバーと同期を行う際に、リモートで失効したライセンスを検出できます。

Low-Level API サンプルアプリ

操作ガイド

1. 装置を起動させて、PCとBluetooth かUSB 接続させて下さい。
2. 装置を装着してください。センサーユニットが眉の上に来るように装着し、髪の毛が挟まらないように注意して下さい。具体的な装着方法については、取扱説明書([リンク](#))をご覧ください。
3. PowerShell を開いていただき、`LowLevelSample.exe`を実行してください。処理がすべて完了すると「Press any key to quit」と表示され、キー入力を受け取るまでアプリは終了しません。これにより、デフォルトのターミナルから実行した場合でもウィンドウが即座に閉じてしまうことを防げます。
4. 実行開始すると、API 初期化、装置検出・選択・接続、（任意）ゲイン調整、（任意）キャリブレーション、計測開始、計測停止、装置切断の順に処理が進行します。

また、装置の選択時とゲイン・キャリブレーション開始時、計測開始時、計測終了時の進行をコマンドインターフェースで進める仕様になっております。

コード全体

このセクションでは、Low-Level API を利用して計測処理を一通り実行するサンプルアプリのコードを掲載しています。

```
using ExBrainSdk;
using ExBrainSdk.Datas;
using ExBrainSdk.Enumerations;

internal class Program
{
    private static void Main()
    {
        var api = new ExBrainApi( (_, s) => Console.WriteLine("C# Status " + s), (id,
message, type) =>
        {
            Console.WriteLine($"C# Log: [{id}] [{type}] {message}");
        });

        Sample(api).Wait();
        api.ExitApi();

        Console.WriteLine("Low-Level Sample finished");
        Console.WriteLine("Press any key to quit");
        _ = Console.ReadKey();
    }

    public static async Task Sample(ExBrainApi api)
    {
```

```

EnmSdkResult sdkResult = api.InitApi(
    needOutputTraceLog: true,
    needOutputCommLog: false, // Not implemented
    logDir: null,
    settingsDir: null, // Not implemented
    tag: "" // Not implemented
);
if (sdkResult != EnmSdkResult.eSuccess)
{
    Console.WriteLine("Failed to initialize API.");
    return;
}

await api.ResetBluetoothModuleAsync();

var searchParam = new ExBrainSearchParameter()
{
    Keywords = ["HOT", "XB"],
    ShouldUSB = true,
    ShouldPairedBluetooth = true,
    ShouldNonPairedBluetooth = true,
    TimeoutSecond = 3
};
api.StartSearchDevice(searchParam);
await Task.Delay(3000);
api.StopSearchDevice();
if (api.DiscoveredDevice.DeviceCount == 0)
{
    Console.WriteLine("No devices found.");
    return;
}
else
{
    IReadOnlyList<DeviceInformationEx> devicesInfo = api.DiscoveredDevice.Devices;
    for (int i = 0; i < devicesInfo.Count; i++)
    {
        Console.WriteLine($"Find Device #{i}: Type: {devicesInfo[i].ConnectType},
DeviceId: {devicesInfo[i].DeviceId}");
    }
    Console.WriteLine("Please select a device index to connect:");
    string? index = await Console.In.ReadLineAsync();
    if (!int.TryParse(index, out int selectedIndex) || selectedIndex < 0 ||
selectedIndex >= devicesInfo.Count)
    {
        Console.WriteLine("Invalid index.");
        return;
    }
}

```

```

    }
    else
    {
        DeviceInformationEx selectedDeviceInfo = devicesInfo[selectedIndex];
        sdkResult = await api.ConnectDeviceAsync(selectedDeviceInfo.DeviceId,
selectedDeviceInfo.ConnectType);
        if (sdkResult != EnmSdkResult.eSuccess)
        {
            Console.WriteLine("Failed to connect to device.");
            return;
        }
    }
}

do
{
    Console.WriteLine("Perform Gain Adjustment? (y/n):");
    string? input = await Console.In.ReadLineAsync();
    if (input?.ToLower() == "y")
    {
        sdkResult = await api.PerformGainAdjustmentAsync(api.HandleId);
        if (sdkResult != EnmSdkResult.eSuccess)
        {
            Console.WriteLine("Gain adjustment failed.");
            await api.DisconnectDeviceAsync();
            return;
        }
        Console.WriteLine("Gain adjustment completed.");
        break;
    }
    else if (input?.ToLower() == "n")
    {
        break;
    }
}
while (true);

do
{
    Console.WriteLine("Perform Calibration? (y/n):");
    string? input = await Console.In.ReadLineAsync();
    if (input?.ToLower() == "y")
    {
        sdkResult = await api.PerformCalibrationAsync(api.HandleId);
        if (sdkResult != EnmSdkResult.eSuccess)
        {

```

```

        Console.WriteLine("Calibration failed.");
        await api.DisconnectDeviceAsync();
        return;
    }
    Console.WriteLine("Calibration completed.");
    break;
}
else if (input?.ToLower() == "n")
{
    break;
}
}
while (true);

EnmSdkMeasurementCycle measurementCycle;
do
{
    Console.WriteLine("Select measurement cycle:");
    Console.WriteLine(" 0: 10Hz");
    Console.WriteLine(" 1: 50Hz");
    Console.WriteLine(" 2: 100Hz");
    Console.WriteLine(" 3: 500Hz");
    string? input = await Console.In.ReadLineAsync();
    if (input == "0")
    {
        measurementCycle = EnmSdkMeasurementCycle.e10Hz;
        break;
    }
    if (input == "1")
    {
        measurementCycle = EnmSdkMeasurementCycle.e50Hz;
        break;
    }
    if (input == "2")
    {
        measurementCycle = EnmSdkMeasurementCycle.e100Hz;
        break;
    }
    if (input == "3")
    {
        measurementCycle = EnmSdkMeasurementCycle.e500Hz;
        break;
    }

    Console.WriteLine("Invalid selection. Please enter 0, 1, 2, or 3.");
}

```

```

while (true);

EnmSdkMeasurementMode measurementMode;
do
{
    Console.WriteLine("Select measurement mode:");
    Console.WriteLine(" 0: Normal");
    Console.WriteLine(" 1: Low Precision / High Reliability");
    Console.WriteLine(" 2: High Precision / Low Reliability");
    string? input = await Console.In.ReadLineAsync();
    if (input == "0")
    {
        measurementMode = EnmSdkMeasurementMode.eNormal;
        break;
    }
    if (input == "1")
    {
        measurementMode = EnmSdkMeasurementMode.eLowPrecisionHighReliability;
        break;
    }
    if (input == "2")
    {
        measurementMode = EnmSdkMeasurementMode.eHighPrecisionLowReliability;
        break;
    }

    Console.WriteLine("Invalid selection. Please enter 0, 1, or 2.");
}
while (true);

List<ExBrainMeasureData> frames = [];
sdkResult = await api.StartMeasureAsync(api.HandleId, "./", (_, data) =>
{
    frames.Add(data);
}, measurementCycle: measurementCycle, measurementMode: measurementMode);
if (sdkResult != EnmSdkResult.eSuccess)
{
    Console.WriteLine("Failed to start measurement.");
    await api.DisconnectDeviceAsync();
    return;
}

do
{
    Console.WriteLine("Stop Measurement? (y/n):");
    string? input = await Console.In.ReadLineAsync();

```

```

        if (input?.ToLower() == "y")
        {
            break;
        }
    }
    while (true);

    sdkResult = await api.StopMeasureAsync(api.HandleId);
    if (sdkResult != EnmSdkResult.eSuccess)
    {
        Console.WriteLine("Failed to stop measurement.");
    }
    await api.DisconnectDeviceAsync();
}
}

```


High-Level API サンプルアプリ

操作ガイド

1. 装置を起動させて、PC と Bluetooth か USB 接続させて下さい。接続ができていないと計測開始の工程へ進められない仕様になっているため、安定しやすい USB 接続を推奨します。
2. 装置を装着してください。センサーユニットが眉の上に来るように装着し、髪の毛が挟まらないように注意して下さい。具体的な装着方法については、取扱説明書（リンク）をご覧ください。
3. PowerShell を開いていただき、`HighLevelSample.exe` を実行してください。処理がすべて完了すると「Press any key to quit」と表示され、キー入力を受け取るまでアプリは終了しません。これにより、デフォルトのターミナルから実行した場合でもウィンドウが即座に閉じてしまうことを防げます。
4. 実行すると、API 初期化、デバイス接続、ゲイン調整、キャリブレーション、実験開始、複数ブロック計測、統計比較、実験終了、デバイス切断までが順に実行されます。

コード全体

このセクションでは、High-Level API を利用して計測処理を一通り実行するサンプルアプリのコードを掲載しています。コードは、デバイス接続からゲイン調整・キャリブレーション、実験の開始、ブロック実行、統計比較、そして計測終了までの流れを段階的に示しており、実際のアプリケーション開発の基盤となる構造を確認できます。

ここで示されるコードは、後続セクション「各パートのコード解説」で詳しく分解しながら解説します。まずは全体像を把握するために、本セクションを参照してください。

Program.cs

```
using ExBrainSdk;
using ExBrainSdk.Enumerations;

var api = new ExBrainApi((_, s) => Console.WriteLine("C# Status " + s), (id, message,
type) =>
{
    Console.WriteLine($"C# Log: [{id}] [{type}] {message}");
});

_ = api.InitApi(true, false, null, null);
if (await api.SearchAndConnectDeviceAsync(EnmSdkConnectType.eNotSelected, 3)
== EnmSdkResult.eSuccess)
{
    await HighLevelSample.Run(api);
    Console.WriteLine("High-Level Sample finished");
}
else
{
    Console.WriteLine("Failed to connect to device.");
}
```

```

}
api.ExitApi();

Console.WriteLine("Press any key to quit");
Console.ReadKey();

```

HighLevelSample.cs

```

using ExBrainSdk;
using ExBrainSdk.HighLevel;

public static class HighLevelSample
{
    public static async Task Run(ExBrainApi api, Action<string>? log =
null, bool performGainAdjustment = true, bool performCalibration = true,
Experiment.NotifyInstantFrameCallback? instantFrameCallback = null, string? outputFolder
= "")
    {
        log ??= Console.WriteLine;

        if (performGainAdjustment)
        {
            _ = await api.PerformGainAdjustmentAsync(api.HandleId);
            log("Gain adjustment completed.");
        }
        if (performCalibration)
        {
            _ = await api.PerformCalibrationAsync(api.HandleId);
            log("Calibration completed.");
        }

        var experiment = new Experiment(api, 10, instantFrameCallback);
        _ = experiment.SetLowpassFilter(2, 0.2, 10);
        _ = await experiment.StartExperimentAsync(outputFolder);
        log("Experiment started. Idling for 5 seconds...");
        await Task.Delay(TimeSpan.FromSeconds(5));
        log("Idling finished");

        int preFittingSeconds = 5;
        int taskSeconds = 5;
        int calmDownSeconds = 5;
        int postFittingSeconds = 5;
        var preTaskMetaData = new MetaData(api, "TestTask", preFittingSeconds, taskSeconds,
calmDownSeconds, postFittingSeconds);
        var postTaskMetaData = new MetaData(api, "TestTask", preFittingSeconds, taskSeconds,

```

```

    calmDownSeconds, postFittingSeconds);
    int totalSeconds = preFittingSeconds + taskSeconds + calmDownSeconds
+ postFittingSeconds;

    string outputDir = outputFolder ?? "";
    log($"Starting block1 ({totalSeconds} seconds)");
    Experiment.NotifyStageStartedCallback callback = stage =>
    {
        switch (stage)
        {
            case BlockStage.PreFitting:
                break;
            case BlockStage.Task:
                log("TASK");
                break;
            case BlockStage.CalmDown:
                log("REST");
                break;
            case BlockStage.PostFitting:
                break;
            default:
                break;
        }
    };
    log("REST");
    Block block1 = await experiment.RunBlockAsync(preTaskMetaData, "TestBlock",
"Participant1", callback);
    log("Block1 finished");
    await File.WriteAllTextAsync(Path.Combine(outputDir,
"block1.json"), block1.ToJson());
    Block block1_reconstructed = Block.FromJson(api, block1.ToJson());
    await File.WriteAllTextAsync(Path.Combine(outputDir,
"block1.csv"), block1_reconstructed.ToCsv());

    log($"Starting block2 ({totalSeconds} seconds)");
    Block block2 = await experiment.RunBlockAsync(preTaskMetaData, "TestBlock",
"Participant1", callback);
    log("Block2 finished");
    await File.WriteAllTextAsync(Path.Combine(outputDir,
"block2.json"), block2.ToJson());

    log($"Starting block3 ({totalSeconds} seconds)");
    Block block3 = await experiment.RunBlockAsync(preTaskMetaData, "TestBlock",
"Participant1", callback);
    log("Block3 finished");
    await File.WriteAllTextAsync(Path.Combine(outputDir,

```

```

"block3.json"), block3.ToJson());

    // Add your own intervention codes.

    log($"Starting block4 ({totalSeconds} seconds)");
    Block block4 = await experiment.RunBlockAsync(postTaskMetaData, "TestBlock",
"Participant1", callback);
    log("Block4 finished");
    await File.WriteAllTextAsync(Path.Combine(outputDir,
"block4.json"), block4.ToJson());

    log($"Starting block5 ({totalSeconds} seconds)");
    Block block5 = await experiment.RunBlockAsync(postTaskMetaData, "TestBlock",
"Participant1", callback);
    log("Block5 finished");
    await File.WriteAllTextAsync(Path.Combine(outputDir,
"block5.json"), block5.ToJson());

    log($"Starting block6 ({totalSeconds} seconds)");
    Block block6 = await experiment.RunBlockAsync(postTaskMetaData, "TestBlock",
"Participant1", callback);
    log("Block6 finished");
    await File.WriteAllTextAsync(Path.Combine(outputDir,
"block6.json"), block6.ToJson());

    _ = await experiment.FinishExperimentAsync();
    await api.DisconnectDeviceAsync();

    var comparison = TaskComparison.CompareWithinParticipantPrePost(api, [block1,
block2, block3], [block4, block5, block6]);
    log($"The statistical test being used is: {comparison.dTHbL_left_testKind}");
    log("Comparison for dTHbL(leftSD3-1cm)Processed");
    log($" P-value = {comparison.dTHbL_left_pValue}, Significant =
{comparison.dTHbL_left_pValue < 0.05}");
    }
}

```

各パートのコード解説

続いて、各パート毎のコード解説を行っていきます。このサンプルでは、トップレベルの `Program.cs` で初期化と接続を行い、その後 `HighLevelSample.Run(...)` で計測処理全体を実行します。

1. Program.cs の内容

```

using ExBrainSdk;
using ExBrainSdk.Enumerations;

var api = new ExBrainApi((_ , s) => Console.WriteLine("C# Status " + s), (id, message,
type) =>
{
    Console.WriteLine($"C# Log: [{id}] [{type}] {message}");
});

_ = api.InitApi(true, false, null, null);
if (await api.SearchAndConnectDeviceAsync(EnmSdkConnectType.eNotSelected, 3)
== EnmSdkResult.eSuccess)
{
    await HighLevelSample.Run(api);
    Console.WriteLine("High-Level Sample finished");
}
else
{
    Console.WriteLine("Failed to connect to device.");
}
api.ExitApi();

Console.WriteLine("Press any key to quit");
Console.ReadKey();

```

1.1 ExBrainApi の生成

最初に [ExBrainApi のコンストラクタ](#) を呼び出し、API インスタンスを生成しています。コンストラクタにはステータス変化通知用コールバックとログ出力用コールバックを渡しており、SDK 内部の状態変化やメッセージをコンソールへ表示できるようにしています。

```

var api = new ExBrainApi((_ , s) => Console.WriteLine("C# Status " + s), (id, message,
type) =>
{
    Console.WriteLine($"C# Log: [{id}] [{type}] {message}");
});

```

これにより、接続や計測の進行状態をサンプル実行中に確認できます。

1.2 API の初期化

次に [InitApi メソッド](#) を呼び出し、SDK を初期化します。このサンプルでは戻り値を利用しないため、discard である `_` に代入しています。

```
_ = api.InitApi(true, false, null, null);
```

引数では、トレースログ出力を有効にし、通信ログ出力や追加のディレクトリ設定は省略しています。実運用アプリケーションでは、戻り値を確認して失敗時のエラー処理を実装することを推奨します。

1.3 デバイスの検索と接続

初期化後は [SearchAndConnectDeviceAsync メソッド](#) を呼び出して、デバイスの検索と接続を行います。

```
if (await api.SearchAndConnectDeviceAsync(EnmSdkConnectType.eNotSelected, 3)
    == EnmSdkResult.eSuccess)
{
    // 計測処理 (1.4 で解説)
}
else
{
    Console.WriteLine("Failed to connect to device.");
}
```

ここでは `EnmSdkConnectType.eNotSelected` を指定しているため、接続方式を固定せず、利用可能な接続手段から接続を試みる構成になっています。第 2 引数 `3` は検索・接続処理に関するタイムアウト秒数です。接続に成功した場合のみ計測処理 (1.4) を実行し、失敗した場合はメッセージを表示します。いずれの場合も後続の終了処理 (1.5) に進み、キー入力を待ってからアプリが終了するため、接続に失敗してもエラーメッセージを確認できます。

1.4 実験本体の実行

デバイス接続が成功した場合 (1.3 の `if` ブロック内)、実験本体である `HighLevelSample.Run(api)` を呼び出します。

```
await HighLevelSample.Run(api);
Console.WriteLine("High-Level Sample finished");
```

ここでは `HighLevelSample.Run(api)` を呼び出すことで、計測に関する一連の処理を実行し、完了後に終了メッセージを表示しています。

1.5 API の終了処理

最後に [ExitApi メソッド](#) を呼び出して API をクリーンに終了します。

```
api.ExitApi();
```

```
Console.WriteLine("Press any key to quit");
```

```
Console.ReadKey();
```

`api.ExitApi()` は接続の成否にかかわらず（1.3 の `if／else` のどちらを通った場合でも）呼び出され、初期化時に確保された内部リソースを解放します。その後 `Console.ReadKey()` でキー入力を待ってからアプリが終了します。これを最後に置くことで、ターミナルから実行した際にウィンドウが即座に閉じてしまうのを防ぎ、接続失敗時のエラーメッセージも確認できるようにしています。

2. HighLevelSample.Run(...) の内容

```
public static async Task Run(ExBrainApi api, Action<string>? log =
null, bool performGainAdjustment = true, bool performCalibration = true,
Experiment.NotifyInstantFrameCallback? instantFrameCallback = null, string? outputFolder
= "")
{
    log ??= Console.WriteLine;

    if (performGainAdjustment)
    {
        _ = await api.PerformGainAdjustmentAsync(api.HandleId);
        log("Gain adjustment completed.");
    }
    if (performCalibration)
    {
        _ = await api.PerformCalibrationAsync(api.HandleId);
        log("Calibration completed.");
    }

    var experiment = new Experiment(api, 10, instantFrameCallback);
    _ = experiment.SetLowpassFilter(2, 0.2, 10);
    _ = await experiment.StartExperimentAsync(outputFolder);
    log("Experiment started. Idling for 5 seconds...");
    await Task.Delay(TimeSpan.FromSeconds(5));
    log("Idling finished");

    int preFittingSeconds = 5;
    int taskSeconds = 5;
    int calmDownSeconds = 5;
    int postFittingSeconds = 5;
    var preTaskMetaData = new MetaData(api, "TestTask", preFittingSeconds, taskSeconds,
calmDownSeconds, postFittingSeconds);
```



```

    var postTaskMetaData = new MetaData(api, "TestTask", preFittingSeconds, taskSeconds,
    calmDownSeconds, postFittingSeconds);
    int totalSeconds = preFittingSeconds + taskSeconds + calmDownSeconds
    + postFittingSeconds;

    string outputDir = outputFolder ?? "";
    log($"Starting block1 ({totalSeconds} seconds)");
    Experiment.NotifyStageStartedCallback callback = stage =>
    {
        switch (stage)
        {
            case BlockStage.PreFitting:
                break;
            case BlockStage.Task:
                log("TASK");
                break;
            case BlockStage.CalmDown:
                log("REST");
                break;
            case BlockStage.PostFitting:
                break;
            default:
                break;
        }
    };
    log("REST");
    Block block1 = await experiment.RunBlockAsync(preTaskMetaData, "TestBlock",
    "Participant1", callback);
    log("Block1 finished");
    await File.WriteAllTextAsync(Path.Combine(outputDir, "block1.json"), block1.ToJson());
    Block block1_reconstructed = Block.FromJson(api, block1.ToJson());
    await File.WriteAllTextAsync(Path.Combine(outputDir, "block1.csv"),
    block1_reconstructed.ToCsv());

    log($"Starting block2 ({totalSeconds} seconds)");
    Block block2 = await experiment.RunBlockAsync(preTaskMetaData, "TestBlock",
    "Participant1", callback);
    log("Block2 finished");
    await File.WriteAllTextAsync(Path.Combine(outputDir, "block2.json"), block2.ToJson());

    log($"Starting block3 ({totalSeconds} seconds)");
    Block block3 = await experiment.RunBlockAsync(preTaskMetaData, "TestBlock",
    "Participant1", callback);
    log("Block3 finished");
    await File.WriteAllTextAsync(Path.Combine(outputDir, "block3.json"), block3.ToJson());

```

```

// Add your own intervention codes.

log($"Starting block4 ({totalSeconds} seconds)");
Block block4 = await experiment.RunBlockAsync(postTaskMetaData, "TestBlock",
"Participant1", callback);
log("Block4 finished");
await File.WriteAllTextAsync(Path.Combine(outputDir, "block4.json"), block4.ToJson());

log($"Starting block5 ({totalSeconds} seconds)");
Block block5 = await experiment.RunBlockAsync(postTaskMetaData, "TestBlock",
"Participant1", callback);
log("Block5 finished");
await File.WriteAllTextAsync(Path.Combine(outputDir, "block5.json"), block5.ToJson());

log($"Starting block6 ({totalSeconds} seconds)");
Block block6 = await experiment.RunBlockAsync(postTaskMetaData, "TestBlock",
"Participant1", callback);
log("Block6 finished");
await File.WriteAllTextAsync(Path.Combine(outputDir, "block6.json"), block6.ToJson());

_ = await experiment.FinishExperimentAsync();
await api.DisconnectDeviceAsync();

var comparison = TaskComparison.CompareWithinParticipantPrePost(api, [block1, block2,
block3], [block4, block5, block6]);
log($"The statistical test being used is: {comparison.dTHbL_left_testKind}");
log("Comparison for dTHbL(leftSD3-1cm)Processed");
log($" P-value = {comparison.dTHbL_left_pValue}, Significant =
{comparison.dTHbL_left_pValue < 0.05}");
}

```

2.1 ログ出力先の決定

関数冒頭では、呼び出し元からログ出力先が渡されていない場合に `Console.WriteLine` を使用するよう設定しています。

```
log ??= Console.WriteLine;
```

これにより、このサンプルはコンソールアプリとしてそのまま使える一方で、GUI アプリケーションやテストコードから別のログ出力先を差し込むこともできます。

2.2 ゲイン調整の実行

`performGainAdjustment` が `true` の場合、[PerformGainAdjustmentAsync メソッド](#) を呼び出してゲイン調整を行います。

```
if (performGainAdjustment)
{
    _ = await api.PerformGainAdjustmentAsync(api.HandleId);
    log("Gain adjustment completed.");
}
```

`api.HandleId` は接続済みデバイスを識別するためのハンドルです。ゲイン調整は、計測値が適切なレンジに収まるよう補正するための前処理として利用されます。このサンプルでは戻り値を検証していませんが、実際のアプリケーションでは結果コードに基づく分岐処理を追加するのが一般的です。

2.3 キャリブレーションの実行

次に `performCalibration` が `true` の場合、[PerformCalibrationAsync メソッド](#) を呼び出してキャリブレーションを行います。

```
if (performCalibration)
{
    _ = await api.PerformCalibrationAsync(api.HandleId);
    log("Calibration completed.");
}
```

キャリブレーションは装着状態や個人差を反映した補正処理です。ゲイン調整と同様に、このサンプルでは正常系の流れを分かりやすく示すため簡潔な記述にしています。

2.4 Experiment インスタンスの生成とフィルタ設定

計測本体は [Experiment クラス](#) によって管理されます。ここでは `Experiment(api, 10, instantFrameCallback)` を生成し、その後ローパスフィルタを設定しています。

```
var experiment = new Experiment(api, 10, instantFrameCallback);
_ = experiment.SetLowpassFilter(2, 0.2, 10);
```

第 2 引数 `10` は SMD 計算用のターゲットウィンドウサイズです。`instantFrameCallback` を指定すれば、計測中のリアルタイムフレーム通知を受け取ることもできます。また、`SetLowpassFilter(2, 0.2, 10)` では Butterworth ローパスフィルタを設定しています。パラメータは順に、フィルタ次数、カットオフ周波数、サンプリングレートを表します。

2.5 実験の開始

フィルタ設定後、[StartExperimentAsync メソッド](#) を呼び出して計測を開始します。

```
_ = await experiment.StartExperimentAsync(outputFolder);
log("Experiment started. Idling for 5 seconds...");
await Task.Delay(TimeSpan.FromSeconds(5));
log("Idling finished");
```

`outputFolder` を指定すると、実験開始時に CSV 出力先として利用されます。開始直後には 5 秒の待機時間を設けており、ここでは計測開始後の安定化期間や、ユーザーによる導入処理の余地を表現しています。

2.6 ブロック時間設定と MetaData の作成

次に、各ブロックで使用する時間設定を定義し、[MetaData クラス](#) を生成します。

```
int preFittingSeconds = 5;
int taskSeconds = 5;
int calmDownSeconds = 5;
int postFittingSeconds = 5;
var preTaskMetaData = new MetaData(api, "TestTask", preFittingSeconds, taskSeconds,
    calmDownSeconds, postFittingSeconds);
var postTaskMetaData = new MetaData(api, "TestTask", preFittingSeconds, taskSeconds,
    calmDownSeconds, postFittingSeconds);
int totalSeconds = preFittingSeconds + taskSeconds + calmDownSeconds + postFittingSeconds;
```

各値は以下の意味を持ちます。

- `preFittingSeconds`: タスク前の事前装着調整フェーズ
- `taskSeconds`: 実タスクフェーズ
- `calmDownSeconds`: タスク後のクールダウンフェーズ
- `postFittingSeconds`: 最後の事後装着調整フェーズ

このサンプルでは前半と後半で同一設定を使用していますが、`preTaskMetaData` と `postTaskMetaData` を分けることで、将来的に条件の異なるブロック構成へ拡張しやすくしています。

2.7 前半ブロックの実行と出力

最初の 3 ブロックは前半条件として実行されます。各ブロックでは [RunBlockAsync メソッド](#) を呼び出し、完了後に JSON ファイルを保存します。1 ブロック目のみ CSV も併せて出力しています。

```
string outputDir = outputFolder ?? "";
log($"Starting block1 ({totalSeconds} seconds)");
Experiment.NotifyStageStartedCallback callback = stage =>
```

```

{
    switch (stage)
    {
        case BlockStage.PreFitting:
            break;
        case BlockStage.Task:
            log("TASK");
            break;
        case BlockStage.CalmDown:
            log("REST");
            break;
        case BlockStage.PostFitting:
            break;
        default:
            break;
    }
};
log("REST");
Block block1 = await experiment.RunBlockAsync(preTaskMetaData, "TestBlock",
"Participant1", callback);
log("Block1 finished");
await File.WriteAllTextAsync(Path.Combine(outputDir, "block1.json"), block1.ToJson());
Block block1_reconstructed = Block.FromJson(api, block1.ToJson());
await File.WriteAllTextAsync(Path.Combine(outputDir, "block1.csv"),
block1_reconstructed.ToCsv());

log($"Starting block2 ({totalSeconds} seconds)");
Block block2 = await experiment.RunBlockAsync(preTaskMetaData, "TestBlock",
"Participant1", callback);
log("Block2 finished");
await File.WriteAllTextAsync(Path.Combine(outputDir, "block2.json"), block2.ToJson());

log($"Starting block3 ({totalSeconds} seconds)");
Block block3 = await experiment.RunBlockAsync(preTaskMetaData, "TestBlock",
"Participant1", callback);
log("Block3 finished");
await File.WriteAllTextAsync(Path.Combine(outputDir, "block3.json"), block3.ToJson());

```

`RunBlockAsync(...)` は、事前装着調整、タスク、クールダウン、事後装着調整を含む 1 ブロック分の処理を完了まで実行し、その結果を [Block クラス](#) として返します。 `ToJson()` と `ToCsv()` を使うことで、ブロック単位の結果をそのまま保存できます。

`block1` では CSV を出力する前に `Block.FromJson(api, block1.ToJson())` で JSON 経由のラウンドトリップを行い、シリアライズ／デシリアライズが正しく機能することを示しています。通常のアプリケーションでは `block1.ToCsv()` を直接呼び出すだけで十分です。

第 4 引数の [NotifyStageStartedCallback](#) は [BlockStage](#) の各フェーズが開始するたびに [PreFitting](#) / [Task](#) / [CalmDown](#) / [PostFitting](#) の順で通知されます。このサンプルでは `callback` という変数に一度だけ定義し、全 6 ブロックで使い回しています。`switch` 文で受け取ったステージごとに分岐し、[Task](#) 開始時（タスク開始時）に [TASK](#)、[CalmDown](#) 開始時（タスク終了時）に [REST](#) をログ出力することで、参加者に対するタスク／休憩の合図を表現しています。block1 の直前に `log("REST")` を出力しているのは、最初のブロック開始までの初期待機状態を [REST](#) として扱うためです。コールバック自体は省略可能ですが、ここでは全ブロックで同一の挙動が必要なため明示的に渡しています。

2.8 介入処理の挿入ポイント

前半と後半のブロック群の間には、コメントで介入処理の挿入ポイントが示されています。

```
// Add your own intervention codes.
```

ここでは、前半ブロック群と後半ブロック群の間に、刺激提示、条件変更、外部タスク開始などの独自処理を差し込むことを想定しています。

2.9 後半ブロックの実行

後半の 3 ブロックも同様に `RunBlockAsync(...)` で実行し、結果を JSON として保存します。

```
log($"Starting block4 ({totalSeconds} seconds)");
Block block4 = await experiment.RunBlockAsync(postTaskMetaData, "TestBlock",
"Participant1", callback);
log("Block4 finished");
await File.WriteAllTextAsync(Path.Combine(outputDir, "block4.json"), block4.ToJson());

log($"Starting block5 ({totalSeconds} seconds)");
Block block5 = await experiment.RunBlockAsync(postTaskMetaData, "TestBlock",
"Participant1", callback);
log("Block5 finished");
await File.WriteAllTextAsync(Path.Combine(outputDir, "block5.json"), block5.ToJson());

log($"Starting block6 ({totalSeconds} seconds)");
Block block6 = await experiment.RunBlockAsync(postTaskMetaData, "TestBlock",
"Participant1", callback);
log("Block6 finished");
await File.WriteAllTextAsync(Path.Combine(outputDir, "block6.json"), block6.ToJson());
```

後半ブロックも前半と同じ `callback` を使い回すことで、[TASK](#) / [REST](#) の合図表示を全ブロックで統一しています。前半ブロック群と後半ブロック群を分けて保存することで、解析時に条件ごとのまとまりを維持しやすくなります。

2.10 実験の終了とデバイス切断

すべてのブロック実行後、[FinishExperimentAsync メソッド](#) で実験を終了し、その後 [DisconnectDeviceAsync メソッド](#) でデバイスを切断します。

```
_ = await experiment.FinishExperimentAsync();
await api.DisconnectDeviceAsync();
```

計測を終了する際には `FinishExperimentAsync()` を呼び出し、その後にデバイス切断処理を行います。

2.11 介入前後の統計比較

最後に、前半 3 ブロックと後半 3 ブロックを [TaskComparison.CompareWithinParticipantPrePost メソッド](#) で比較します。

```
var comparison = TaskComparison.CompareWithinParticipantPrePost(api, [block1, block2,
block3], [block4, block5, block6]);
log($"The statistical test being used is: {comparison.dTHbL_left_testKind}");
log("Comparison for dTHbL(leftSD3-1cm)Processed");
log($" P-value = {comparison.dTHbL_left_pValue}, Significant =
{comparison.dTHbL_left_pValue < 0.05}");
```

この比較は、同一参加者の介入前後ブロック群を対象にした統計評価です。ログには使用された検定種別と `dTHbL(leftSD3-1cm)Processed` の p 値、有意差判定結果を出力しています。必要に応じて右側データや差分値の出力を追加することも可能です。

以上で、High-Level API サンプルアプリのソースコード解説を終わります。

チュートリアル：デバイスの検索から計測データ取得まで

このチュートリアルでは、サンプルコード `tutorial.cs` を例に、Low-Level API を使って以下のステップを順に実装する方法を説明します。

- 1. API の初期化
- 2. デバイスの検索と接続
- 3. ゲイン調整
- 4. キャリブレーション
- 5. 計測の開始とデータ受信
- 6. 計測の停止と切断

各ステップのコードは `tutorial.cs` の該当部分の抜粋です。末尾の「[コード全体](#)」に `tutorial.cs` の完全なコードを掲載しています。

コンソール出力のプレフィックス

このチュートリアルコードでは、コンソール出力に以下のプレフィックスを付けてメッセージの種類を識別できるようにしています。

プレフィックス	用途
[SDK]	SDK からの状態変化通知
[LOG]	SDK 内部ログ
[OK]	処理成功
[ERROR]	処理失敗
[DATA]	計測データ出力
なし	ユーザーへの案内・操作指示

前提条件

- .NET 8.0 以上
- ExBrain SDK の NuGet パッケージを参照済み
- デバイスが PC と Bluetooth または USB で接続済み（Bluetooth の場合はペアリング済み）

起動方法

SDK の Tutorial フォルダに本チュートリアルプログラム (`tutorial.cs`) が掲載されています。.NET がインストールされている環境では、Tutorial フォルダで以下のコマンドを実行すると `tutorial.cs` を起動できます。

```
dotnet run
```

ステップ 1: API の初期化

```
// ▼ Main() の中
var api = new ExBrainApi(
    notifyStatusChangedCallBack: (id, status) => Console.WriteLine($"[SDK] ID: {id}
    Status: {status}"),
    notifyProcessMessageCallBack: (id, message, type) => Console.WriteLine($"[LOG] ID: {id}
    [{type}] {message}")
);

// ▼ Run() の中
EnmSdkResult sdkResult = api.InitApi(
    needOutputTraceLog: true,
    needOutputCommLog: false,
    logDir: null,           // ログファイルの出力先フォルダ。null の場合はカレントディレクトリ
    settingsDir: null,     // 設定ファイルの読み込み先 (未実装)
    tag: ""                // 識別タグ (未実装)
);
if (sdkResult != EnmSdkResult.eSuccess)
{
    Console.WriteLine("[ERROR] API の初期化に失敗しました。");
    return;
}
```

`logDir` にフォルダパスを指定するとトレースログファイルが出力されます。`null` の場合はカレントディレクトリに出力されま

ステップ 2: デバイスの検索と接続

```
await api.ResetBluetoothModuleAsync();

api.StartSearchDevice(new ExBrainSearchParameter
{
    Keywords = ["HOT", "XB"],
    ShouldUSB = true,
    ShouldPairedBluetooth = true,
    ShouldNonPairedBluetooth = true,
```

```

        TimeoutSecond = 3
    });
    await Task.Delay(TimeSpan.FromSeconds(3));
    api.StopSearchDevice();

    if (api.DiscoveredDevice.DeviceCount == 0)
    {
        Console.WriteLine("[ERROR] デバイスが見つかりませんでした。");
        return;
    }

    // 検出されたデバイスを一覧表示する
    IReadOnlyList<DeviceInformationEx> devices = api.DiscoveredDevice.Devices;
    Console.WriteLine("検出されたデバイス一覧:");
    for (int i = 0; i < devices.Count; i++)
    {
        Console.WriteLine($" 番号 {i}: 種類={devices[i].ConnectType}
ID={devices[i].DeviceId}");
    }

    // 接続するデバイスを選択する
    Console.WriteLine($"接続する番号を入力してください (0 ~ {devices.Count - 1}):");
    string? indexInput = await Console.In.ReadLineAsync();
    if (!int.TryParse(indexInput, out int selectedIndex) || selectedIndex < 0 || selectedIndex
    >= devices.Count)
    {
        Console.WriteLine("[ERROR] 番号が正しくありません。");
        return;
    }

    sdkResult = await api.ConnectDeviceAsync(devices[selectedIndex].DeviceId,
    devices[selectedIndex].ConnectType);
    if (sdkResult != EnmSdkResult.eSuccess)
    {
        Console.WriteLine("[ERROR] 接続に失敗しました。");
        return;
    }

    Console.WriteLine("[OK] 接続しました。");

```

ステップ 3: ゲイン調整

ゲイン調整は光センサーの出力レベルを最適化する処理です。計測前に毎回実施してください。

```
// 装着確認
Console.WriteLine("デバイスを額に装着してください。");
Console.WriteLine("センサーユニットが眉の上に来るように装着し、髪の毛が挟まらないように注意してください。");
Console.WriteLine("装着が完了したら Enter キーを押してください。");
_ = await Console.In.ReadLineAsync();

// ゲイン調整
Console.WriteLine("ゲイン調整を開始します。準備ができたなら Enter キーを押してください。");
_ = await Console.In.ReadLineAsync();
sdkResult = await api.PerformGainAdjustmentAsync(api.HandleId);
if (sdkResult != EnmSdkResult.eSuccess)
{
    Console.WriteLine("[ERROR] ゲイン調整に失敗しました。");
    await api.DisconnectDeviceAsync();
    return;
}
Console.WriteLine("[OK] ゲイン調整が完了しました。");
```

ステップ 4: キャリブレーション

キャリブレーションは `dThbL` のベースラインをゼロに補正する処理です。

```
// キャリブレーション
Console.WriteLine("キャリブレーションを開始します。準備ができたなら Enter キーを押してください。");
_ = await Console.In.ReadLineAsync();
sdkResult = await api.PerformCalibrationAsync(api.HandleId);
if (sdkResult != EnmSdkResult.eSuccess)
{
    Console.WriteLine("[ERROR] キャリブレーションに失敗しました。");
    await api.DisconnectDeviceAsync();
    return;
}
Console.WriteLine("[OK] キャリブレーションが完了しました。");
```

ステップ 5: 計測の開始とデータ受信

計測データは 10Hz（1秒に10回）でコールバックされます。毎フレーム出力するとコンソールが流れるため、ここでは5フレームに1回表示します。

```
Console.WriteLine("計測を開始します。計測中は Enter キーで停止できます。");
Console.WriteLine("準備ができたなら Enter キーを押してください。");
```

```

_ = await Console.In.ReadLineAsync();

var receivedData = new List<ExBrainMeasureData>();
int frameCount = 0;

sdkResult = await api.StartMeasureAsync(
    handleId: api.HandleId,
    measureLogFolder: "./",    // CSV ログをカレントディレクトリに出力する
    callback: (_, data) =>
    {
        receivedData.Add(data);
        frameCount++;

        // 5フレームに1回だけコンソールに出力する
        if (frameCount % 5 == 0)
        {
            Console.WriteLine(
                $"[DATA][{frameCount}] " +
                $"左 1cm: {data.dTHbL_leftSD1cm:F4} " +
                $"左 3cm: {data.dTHbL_leftSD3cm:F4} " +
                $"右 1cm: {data.dTHbL_rightSD1cm:F4} " +
                $"右 3cm: {data.dTHbL_rightSD3cm:F4}"
            );
        }
    },
    measurementCycle: EnmSdkMeasurementCycle.e10Hz,
    measurementMode: EnmSdkMeasurementMode.eNormal
);

if (sdkResult != EnmSdkResult.eSuccess)
{
    Console.WriteLine("[ERROR] 計測の開始に失敗しました。");
    await api.DisconnectDeviceAsync();
    return;
}

// Enter キーが押されるまで計測を継続する
_ = Console.ReadLine();

```

`measureLogFolder` にフォルダパスを指定すると `HOTLog_YYYYMMDD_HHMMSS.csv` が自動出力されます。CSV が不要な場合は `null` を渡してください。

コールバック内で受け取れる主なデータは以下のとおりです。

プロパティ	内容
dTHbL_leftSD1cm	左チャンネル 1cm のヘモグロビン濃度変化
dTHbL_leftSD3cm	左チャンネル 3cm のヘモグロビン濃度変化
dTHbL_rightSD1cm	右チャンネル 1cm のヘモグロビン濃度変化
dTHbL_rightSD3cm	右チャンネル 3cm のヘモグロビン濃度変化
SamplingMotion	モーションセンサーデータ（加速度・ジャイロ）
SamplingBattery	バッテリー情報

注意: v3.1.0 では `EnmSdkMeasurementCycle.e10Hz` と `EnmSdkMeasurementMode.eNormal` の組み合わせのみ対応しています。

ステップ 6: 計測の停止と切断

```

sdkResult = await api.StopMeasureAsync(api.HandleId);
if (sdkResult != EnmSdkResult.eSuccess)
{
    Console.WriteLine("[ERROR] 計測の停止に失敗しました。");
}

await api.DisconnectDeviceAsync();
Console.WriteLine($"[OK] 計測終了。受信フレーム数: {receivedData.Count}");

```

`api.ExitApi()` はコード全体の `Main()` で呼び出されます。

コード全体（tutorial.cs）

```

using ExBrainSdk;
using ExBrainSdk.Datas;
using ExBrainSdk.Enumerations;

internal class Program
{
    private static void Main()
    {
        var api = new ExBrainApi(

```

```

        notifyStatusChangedCallBack: (id, status) => Console.WriteLine($"[SDK] ID: {id}
Status: {status}"),
        notifyProcessMessageCallBack: (id, message, type) => Console.WriteLine($"[LOG]
ID: {id} [{type}] {message}")
    );

    Run(api).Wait();
    api.ExitApi();

    Console.WriteLine("Press any key to quit");
    _ = Console.ReadKey();
}

public static async Task Run(ExBrainApi api)
{
    // 初期化
    EnmSdkResult sdkResult = api.InitApi(
        needOutputTraceLog: true,
        needOutputCommLog: false,
        logDir: null,
        settingsDir: null,
        tag: ""
    );
    if (sdkResult != EnmSdkResult.eSuccess)
    {
        Console.WriteLine("[ERROR] API の初期化に失敗しました。");
        return;
    }

    await api.ResetBluetoothModuleAsync();

    // デバイス検索
    api.StartSearchDevice(new ExBrainSearchParameter
    {
        Keywords = ["HOT", "XB"],
        ShouldUSB = true,
        ShouldPairedBluetooth = true,
        ShouldNonPairedBluetooth = true,
        TimeoutSecond = 3
    });
    await Task.Delay(TimeSpan.FromSeconds(3));
    api.StopSearchDevice();

    if (api.DiscoveredDevice.DeviceCount == 0)
    {
        Console.WriteLine("[ERROR] デバイスが見つかりませんでした。");
    }
}

```

```

        return;
    }

    // 接続するデバイスを選択する
    IReadOnlyList<DeviceInformationEx> devices = api.DiscoveredDevice.Devices;
    Console.WriteLine("検出されたデバイス一覧:");
    for (int i = 0; i < devices.Count; i++)
    {
        Console.WriteLine($" 番号 {i}: 種類={devices[i].ConnectType}
ID={devices[i].DeviceId}");
    }
    Console.WriteLine($"接続する番号を入力してください (0 ~ {devices.Count - 1}):");
    string? indexInput = await Console.In.ReadLineAsync();
    if (!int.TryParse(indexInput, out int selectedIndex) || selectedIndex < 0 ||
selectedIndex >= devices.Count)
    {
        Console.WriteLine("[ERROR] 番号が正しくありません。");
        return;
    }

    sdkResult = await api.ConnectDeviceAsync(devices[selectedIndex].DeviceId,
devices[selectedIndex].ConnectType);
    if (sdkResult != EnmSdkResult.eSuccess)
    {
        Console.WriteLine("[ERROR] 接続に失敗しました。");
        return;
    }
    Console.WriteLine("[OK] 接続しました。");

    // 装着確認
    Console.WriteLine("デバイスを額に装着してください。");
    Console.WriteLine("センサーユニットが眉の上に来るように装着し、髪の毛が挟まらないように注意
してください。");
    Console.WriteLine("装着が完了したら Enter キーを押してください。");
    _ = await Console.In.ReadLineAsync();

    // ゲイン調整
    Console.WriteLine("ゲイン調整を開始します。準備ができたなら Enter キーを押してください。");
    _ = await Console.In.ReadLineAsync();
    sdkResult = await api.PerformGainAdjustmentAsync(api.HandleId);
    if (sdkResult != EnmSdkResult.eSuccess)
    {
        Console.WriteLine("[ERROR] ゲイン調整に失敗しました。");
        await api.DisconnectDeviceAsync();
        return;
    }

```

```

Console.WriteLine("[OK] ゲイン調整が完了しました。");

// キャリブレーション
Console.WriteLine("キャリブレーションを開始します。準備ができたなら Enter キーを押してくだ
さい。");
_ = await Console.In.ReadLineAsync();
sdkResult = await api.PerformCalibrationAsync(api.HandleId);
if (sdkResult != EnmSdkResult.eSuccess)
{
    Console.WriteLine("[ERROR] キャリブレーションに失敗しました。");
    await api.DisconnectDeviceAsync();
    return;
}
Console.WriteLine("[OK] キャリブレーションが完了しました。");

// 計測開始確認
Console.WriteLine("計測を開始します。計測中は Enter キーで停止できます。");
Console.WriteLine("準備ができたなら Enter キーを押してください。");
_ = await Console.In.ReadLineAsync();

var receivedData = new List<ExBrainMeasureData>();
int frameCount = 0;

sdkResult = await api.StartMeasureAsync(
    handleId: api.HandleId,
    measureLogFolder: "./",
    callback: (_, data) =>
    {
        receivedData.Add(data);
        frameCount++;
        if (frameCount % 5 == 0)
        {
            Console.WriteLine(
                $"[DATA][{frameCount}] " +
                $"左 1cm: {data.dTHbL_leftSD1cm:F4} " +
                $"左 3cm: {data.dTHbL_leftSD3cm:F4} " +
                $"右 1cm: {data.dTHbL_rightSD1cm:F4} " +
                $"右 3cm: {data.dTHbL_rightSD3cm:F4}"
            );
        }
    },
    measurementCycle: EnmSdkMeasurementCycle.e10Hz,
    measurementMode: EnmSdkMeasurementMode.eNormal
);

if (sdkResult != EnmSdkResult.eSuccess)

```



```

{
    Console.WriteLine("[ERROR] 計測の開始に失敗しました。");
    await api.DisconnectDeviceAsync();
    return;
}

_ = Console.ReadLine();

// 計測停止・切断
sdkResult = await api.StopMeasureAsync(api.HandleId);
if (sdkResult != EnmSdkResult.eSuccess)
{
    Console.WriteLine("[ERROR] 計測の停止に失敗しました。");
}

await api.DisconnectDeviceAsync();
Console.WriteLine($"[OK] 計測終了。受信フレーム数: {receivedData.Count}");
}
}

```

出力CSVデータ

CSVデータの項目について

計測が終了するとCSVデータが出力されます。

Low-Level API による基本データを記録したCSVファイルと、High-Level API による解析結果を含むCSVファイルとの2種類が存在します。

それぞれのCSVファイルにおける、データ項目について説明します。

Low-Level CSV

Low-Level APIから出力されるCSVデータの項目です。

計測モードによるデータの違い

計測サイクル（10Hz / 50Hz / 100Hz / 500Hz）によって、1フレームあたりのデータ内容が異なります。
（現在、ExBrainSDK v3.1.0では10Hzモードのみ対応しています。）

- 10Hz モード: 全項目が出力されます。
- 50Hz / 100Hz / 500Hz モード（高速サンプリングモード：現在、ExBrainSDK v3.1.0では未対応です。）：各フレームには左チャンネルの生体光センサーデータのみが含まれます。右チャンネルデータ・IMUデータ・飽和カウンタ・カラーインデックス・心拍数・ノイズ検出フラグは常に 0 となります。BatteryVolt / BatteryRemain は計測開始時点の値が全フレームに固定で記録されます。また、このモードではCSVファイル名の末尾に `_fast` が付加されます。

下表の「高速サンプリング」列は、50Hz / 100Hz / 500Hz モードでの出力可否を示します（○: 有効値あり、0: 常に0）。

項目名	説明	高速サンプリング
DeviceTime	計測時の年月日時分秒.ミリ秒 [yyyy/MM/dd HH:mm:ss.fff]	有効
HeadsetTime(sec)	開始時刻からの経過時間 [秒]	有効
BrainIndex(left)	脳活動値（左） [-]	有効
BrainIndex(right)	脳活動値（右） [-]	常に0
PulseRate	心拍数 [bpm]	常に0
dTHbL(leftSD1cm)	ヘモグロビン濃度変化（左 SD1cm） [-]	有効
dTHbL(leftSD3cm)	ヘモグロビン濃度変化（左 SD3cm） [-]	有効

項目名	説明	高速サンプリング
dTHbL(rightSD1cm)	ヘモグロビン濃度変化（右 SD1cm） [-]	常に0
dTHbL(rightSD3cm)	ヘモグロビン濃度変化（右 SD3cm） [-]	常に0
Saturation(leftSD1cm)	飽和カウンタ（左 SD1cm） [回]	常に0
Saturation(leftSD3cm)	飽和カウンタ（左 SD3cm） [回]	常に0
Saturation(rightSD1cm)	飽和カウンタ（右 SD1cm） [回]	常に0
Saturation(rightSD3cm)	飽和カウンタ（右 SD3cm） [回]	常に0
ColorIndex(left)	カラーインデックス（左） [-]	常に0
ColorIndex(right)	カラーインデックス（右） [-]	常に0
NoiseDetectionFlag	ノイズ検出フラグ [-]	常に0
Mark	マーカー管理機能により、被験者の状態やタスクシーケンスの内容に関する結果を記録する。項目間を\ で結合させています。	有効
Acc-X	加速度 X [G]	常に0
Acc-Y	加速度 Y [G]	常に0
Acc-Z	加速度 Z [G]	常に0
Gyro-X	角速度 X [dps]	常に0
Gyro-Y	角速度 Y [dps]	常に0
Gyro-Z	角速度 Z [dps]	常に0
BatteryVolt	バッテリー電圧 [V]	有効 (計測開始時の値で固定)
BatteryRemain	バッテリー残量 [%]	有効 (計測開始時の値で固定)

High-Level CSV

High-Level APIから出力されるCSVデータの項目です。Low-Level CSVとは独立したファイルで、解析済みデータを含みます。

項目名	説明
DeviceTime	計測時の年月日時分秒.ミリ秒 [yyyy/MM/dd HH:mm:ss.fff]
HeadsetTime(sec)	開始時刻からの経過時間 [秒]
BlockName	ブロック名
Identity	識別子
StageName	ステージ名（PreFitting / Task / CalmDown / PostFitting）
BrainIndex(left)	脳活動値（左） [-]
BrainIndex(right)	脳活動値（右） [-]
ColorIndex(left)	カラーインデックス（左） [-]
ColorIndex(right)	カラーインデックス（右） [-]
PulseRate	心拍数 [bpm]
dTHbL(leftSD1cm)	ヘモグロビン濃度変化（左 SD1cm） [-]
dTHbL(leftSD3cm)	ヘモグロビン濃度変化（左 SD3cm） [-]
dTHbL(rightSD1cm)	ヘモグロビン濃度変化（右 SD1cm） [-]
dTHbL(rightSD3cm)	ヘモグロビン濃度変化（右 SD3cm） [-]
SyntheticAcc	合成加速度 [G]
Acc-X	加速度 X [G]
Acc-Y	加速度 Y [G]
Acc-Z	加速度 Z [G]
Gyro-X	角速度 X [dps]
Gyro-Y	角速度 Y [dps]
Gyro-Z	角速度 Z [dps]
ElapsedSec	カウント開始からの経過時間 [秒]

項目名	説明
DropCount	ハードウェア側カウンタについて、現在のと前回の差から1を引いた値（ハードウェア側でドロップされたフレーム数のみを考慮）。[フレーム]
Mark	マーカー管理機能により、被験者の状態やタスクシーケンスの内容に関する結果を記録する。項目間を\ で結合させています。
dTHbL(leftSD3-1cm)Processed	処理済みヘモグロビン濃度変化（左 SD3-1cm） [-]
dTHbL(rightSD3-1cm)Processed	処理済みヘモグロビン濃度変化（右 SD3-1cm） [-]
SmdBrainIndex(left)	標準化平均差・脳活動値（左）(Standardized Mean Difference) [-]
SmdBrainIndex(right)	標準化平均差・脳活動値（右）(Standardized Mean Difference) [-]

トラブルシューティング

Q&A

Bluetoothでそもそも装置が検出されない場合は？

確認事項：

- Bluetooth機能はONになっていますか？
- 装置が他のBluetooth機器とペアリングされていませんか？
- 他のアプリケーションが、既に装置とペアリングを行っていませんか？
- お使いのPCはBluetoothに対応していますか？

対処方法：

- 装置の電源を切り、次にアプリケーションを完全に終了し、「設定」から「Bluetoothの設定」を確認し、装置がペアリング前の状態でリストに出ていることを確認します。
 - 既にペアリング済みとなっている場合は、一度接続を切断してからアプリケーションを立ち上げ直してください。
-

Bluetooth接続が頻繁に切れる場合は？

確認事項：

- 周囲の電波状況は安定していますか？ Wi-Fi電波やBluetooth機器がたくさん電波を発している環境ではペアリングが頻繁に切れる可能性があります。
- 電源が切れていたり、バッテリーの残量が少なくなっていないですか？

対処方法：

- 電波環境の良い場所に移動します。もしくは、計測に不必要なネットワーク機器を使用しないようにします。
 - バッテリー残量が少ない場合は充電してください。
 - 電源が切れているときは、モバイルバッテリーやAC アダプター等を接続し計測することも可能です。
-

ゲイン調整で何度も失敗してしまう場合は？

確認事項：

- ヘッドセットは正しく装着されていますか？ 髪の毛がはさまっていたり、緩く取り付けられていたり、汗を多くかいていたりとゲイン調整に失敗することがあります。

対処方法：

- ヘッドセットを正しく装着します。稀に前額部の形状によっては、センサーユニットが額に密着しない方がいらっしゃいます。センサーユニットの位置を微調整する等して、密着しやすい箇所での計測をお試しください。
-

サポート窓口

上記以外のケースで解決されない場合は、大変お手数ですが下記窓口までお問合せいただければと思います。

- [お問い合わせフォーム（株式会社NeU）](#) 

NIRSハードウェア & システム（研究用機器）を選択してください。

使用許諾

本SDKの使用条件およびライセンスに関する詳細については、ExBrainSDK に付属の LICENSE.txt を参照してください。

その他

参考資料

本SDKをご利用になるうえで、関連する参考資料になります。

- 取扱説明書 研究開発向け 携帯型脳活動計測装置(型式：HOT-2000)

サードパーティ利用状況

本SDKは以下のサードパーティライブラリを使用しています。

Ring Buffer

- リポジトリ: <https://github.com/jnk0le/Ring-Buffer>
- 著作権: Copyright (c) 2017-2022 Jan Oleksiewicz
- ライセンス: MIT License

内部のデータ通信に使用するリングバッファの実装です。C++11のみで実装されており、OS依存がありません。ロックフリー・ウェイトフリーなSPSC（Single Producer Single Consumer）動作をサポートし、動的メモリ確保や例外を一切使用しない軽量な実装です。

MITライセンスの全文は以下の通りです。

MIT License

Copyright (c) 2022 (2017-2022 @CC0) Jan Oleksiewicz

Permission is hereby granted, free of charge, to any person obtaining a copy of this software and associated documentation files (the "Software"), to deal in the Software without restriction, including without limitation the rights to use, copy, modify, merge, publish, distribute, sublicense, and/or sell copies of the Software, and to permit persons to whom the Software is furnished to do so, subject to the following conditions:

The above copyright notice and this permission notice shall be included in all copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.

Simple-FFT

- リポジトリ: <https://github.com/d1vanov/Simple-FFT>
- 著作権: Copyright (c) 2013-2020 Dmitry Ivanov
- ライセンス: MIT License

脳波データの周波数解析に使用する高速フーリエ変換（FFT）の実装です。ヘッダーオンリーのC++ライブラリで、1次元・2次元・3次元データのFFTに対応しています。Radix-2 Cooley-Tukey アルゴリズムを採用しており、外部依存なしで利用できます。なお、変換対象のデータサイズは2の累乗である必要があります。

MITライセンスの全文は以下の通りです。

Copyright (c) 2013-2020 Dmitry Ivanov

The MIT License (MIT)

Permission is hereby granted, free of charge, to any person obtaining a copy of this software and associated documentation files (the "Software"), to deal in the Software without restriction, including without limitation the rights to use, copy, modify, merge, publish, distribute, sublicense, and/or sell copies of the Software, and to permit persons to whom the Software is furnished to do so, subject to the following conditions:

The above copyright notice and this permission notice shall be included in all copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.